Months 16 to 18: MLOps

Senior Data Scientist roadmap

B27-L1 The model registry, versions and aliases

Register every candidate as a version, serve through an alias, resolve the alias once, and keep old versions as rollback targets.

```
info = mlflow.sklearn.log_model(model, name="model")       # info.model_uri: "models:/m-..."
version = mlflow.register_model(info.model_uri, "survey_topics").version   # 1, then 2, ...
client = MlflowClient()
client.set_registered_model_alias("survey_topics", "champion", "1")
model = mlflow.pyfunc.load_model("models:/survey_topics@champion")
```

Registering a model (MLflow 3.17)	mlflow.register_model(model_uri, name) creates version 1, 2, ... of a named model.
Alias	A movable name for one version. set_registered_model_alias(name, alias, version); load models:/name@alias.
Promote and roll back	Both move the alias. Keep old versions as rollback targets.
Resolve once	get_model_version_by_alias, then load models:/name/version, so the model and the reported version agree.

B27-L2 Serving a model with FastAPI

Let pydantic schemas validate requests, load the model once in a lifespan function inside an app factory, and test with TestClient as a context manager.

```
class PredictRequest(BaseModel):
    texts: list[Annotated[str, Field(min_length=1, max_length=2_000)]] = Field(min_length=1)

@app.post("/predict", response_model=PredictResponse)
def predict(request: PredictRequest) -> PredictResponse: ...
```

```
@asynccontextmanager
async def lifespan(app: FastAPI):
    app.state.model, app.state.meta = loader()
    yield
```

Request schema (FastAPI 0.162, pydantic 2.14)	A pydantic model with Field rules. Invalid input gets 422 with a detail list before your function runs.
Loading the model	Once, in a lifespan function that stores it on app.state. Not per request, not at import time.
Status codes	503 from /health until the model loads, 422 for invalid input, 413 for too many texts.
TestClient	with TestClient(app) as client: runs the lifespan. Without with, start-up never runs.

B27-L3 Packaging a model service as a container image

Pin serving dependencies, install them before copying code, bind to 0.0.0.0, run as a non-root user, and read settings from the environment at start-up.

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements-serve.txt .
```

```
RUN pip install --no-cache-dir -r requirements-serve.txt
COPY src/ src/
COPY model/ model/
RUN useradd --create-home appuser
USER appuser
EXPOSE 8000
CMD ["uvicorn", "serve_lab.app:build", "--factory", "--host", "0.0.0.0", "--port", "8000"]
```

Pinned serving requirements	name==version for each package the service needs, and nothing else. The model's scikit-learn version must match.
Dockerfile order	FROM, copy requirements, install, copy code and model, non-root USER, CMD. Docker reuses steps until an input changes.
--host 0.0.0.0	Uvicorn 0.54 defaults to 127.0.0.1, which only the container itself can reach.
Settings and secrets	Read settings from environment variables and check them at start-up. Secrets are injected at run time, never built in.

B27-L4 Deploying the container to Azure Container Apps

Deploy with az containerapp up, keep secrets in Key Vault references passed as secretref variables, probe /health over HTTP, and choose min replicas for your latency needs.

```
az login
az extension add --name containerapp --upgrade
az containerapp up --name survey-topics --source . --ingress external --target-port 8000
```

az containerapp up --source .	Creates the resource group, environment, Log Analytics and a registry, builds from the Dockerfile, deploys and prints the URL.
Secrets in Container Apps	--secrets name=value or a Key Vault reference; --env-vars NAME=secretref:name. A secret change needs a restart or a new revision.
Probes	Startup, liveness, readiness. HTTP passes on 200 to 399. The default with ingress is TCP, which passes before the model loads.
Default scaling	HTTP rule, 0 to 10 replicas, 10 concurrent requests each. Scale to zero costs nothing but adds a cold start.

B27-L5 Monitoring a served model for drift

Log structured records without raw text, compare inputs and outputs with a fixed-bin PSI, treat 0.1 and 0.25 as starting points, and confirm drift with labels.

```
psi = sum((a - e) * ln(a / e))      # over bins
```

What to monitor without labels	Input distributions, predicted class shares, confidence, error rate and p95 latency.
PSI	sum((a - e) * ln(a / e)) over the reference's quantile bins, with shares floored at a small value.
PSI bands	Below 0.1 stable, 0.1 to 0.25 moderate, above 0.25 large: a rule of thumb from credit scoring. Tune it to your data.
Prediction logs	One structured record per request with version, counts, confidence and latency. Never the respondent's text.

B27-L6 Retraining, promotion and rollback

Compare champion and challenger on the same recent data, shadow before switching, promote by moving an alias and deploying a new revision, and keep old versions, images and revisions for rollback.

Champion and challenger	Compare on the same recent labelled batches. Promote only for a gain above a margin, won on most batches.
Shadow mode	Serve the champion, run the challenger on the same requests, log both, and review agreement and swaps.
Promotion	Move the alias, export, build an image tagged with the version, deploy a new revision.
Rollback	Move the alias back and return traffic to the old revision, or redeploy the old image tag. Keep old versions, tags and revisions.